AI PERSONAL STUDY ASSISTANT

Python Programming

Study Material for Beginners

First-Year B.Tech — Computer Science and Engineering (AI & ML)

Project	Generative AI by IBM
Subject	Python Programming
Student Name	Pooja Biju
Department	Computer Science and Engineering (AI & ML)

A self-contained introduction to Python concepts, examples, practice, and revision.

Designed for learning, quick revision, and question-answer retrieval by a study chatbot.

1. Introduction to Python

What is Python? Python is a high-level, general-purpose programming language. A program is a set of instructions that tells a computer what to do. Python's readable syntax makes it friendly for beginners while still being useful for professional software.

History and development

Python was created by Guido van Ross-um and first released in 1991. It was designed to make programs easier to read and write. Python 3, the modern version used in this material, was released in 2008 and continues to evolve through a community-led process.

Important features

• **Readable syntax:** code often resembles simple English.

• **Interpreted:** Python code is executed by a Python interpreter.

• **High-level:** many low-level computer details are handled for you.

• **Cross-platform:** Python runs on Windows, macOS, and Linux.

• **Large standard library:** many useful tools come with Python.

• **Multi-paradigm:** supports procedural, object-oriented, and functional styles.

• **Open source:** it can be used and shared under an open-source licence.

Applications in real life

Area	Example use
Web development	Building websites and web APIs
Data science	Cleaning, analysing, and visualising data
AI and machine learning	Training models and building intelligent applications
Automation	Renaming files or repeating routine computer tasks
Scientific computing	Numerical calculations and simulations
Education	Learning programming and computational thinking

Advantages of learning Python

• Beginners can focus on problem-solving instead of complicated syntax.

• Python is useful across several computing fields, including AI and data science.

• Its large community provides tutorials, documentation, and libraries.

• The same basic skills can be used for small scripts and larger applications.

Remember: Python is a programming language, not a single application. A Python program is a sequence of instructions written using Python syntax.

2. Python Fundamentals

Variables are names that refer to values. Python creates a variable when a value is assigned with =. A **constant** is a value intended not to change; Python does not enforce constants, so programmers commonly write their names in uppercase.

```
student_name = "Anu"
age = 18
pi_value = 3.14159
is_student = True
print(student_name)
print(age)
print(pi_value)
print(is_student)
```

Expected output:

```
Anu<br/>18<br/>3.14159<br/>True
```

Line by line: each assignment stores a value under a name. `print()` displays the value. Text is written inside quotes; numbers do not need quotes.

Basic data types

Type	Meaning	Example
int	Whole number	count = 12
float	Number with a decimal part	temperature = 28.5
str	Text (string)	course = "Python"
bool	Truth value: True or False	passed = True

Keywords and identifiers

Keywords are reserved words with special meanings, such as `if`, `else`, `for`, `while`, `def`, `return`, `True`, and `False`. Do not use keywords as variable names.

An **identifier** is a name given to a variable, function, or other program item.

Rules for naming variables

• Use letters, digits, and underscores; do not start with a digit.

• Names are case-sensitive: `score` and `Score` are different.

• Do not use spaces or punctuation such as hyphens in a name.

• Avoid Python keywords. Choose clear names such as `total_marks`.

A small example

```
marks = 80
bonus = 5
total = marks + bonus
print("Total:", total)
```

Expected output:

```
Total: 85
```

The program adds `marks` and `bonus`, stores the result in `total`, and prints a label followed by the result.

3. Operators in Python

An **operator** performs an operation on one or more values. The values are called operands.

Arithmetic operators

Operator	Meaning	Example result (a=10, b=3)
+	Addition	a + b → 13
-	Subtraction	a - b → 7
*	Multiplication	a * b → 30
/	Division (returns a float)	a / b → 3.333...
//	Floor division	a // b → 3
%	Remainder	a % b → 1
**	Power	a ** b → 1000

```
a = 10
b = 3
print(a + b, a // b, a % b, a ** b)
```

Expected output:

```
13 3 1 1000
```

Comparison operators

Comparisons produce a Boolean value: `True` or `False`. Operators include == (equal), `!=` (not equal), >, <, >=, and <=.

```
x = 7
print(x == 7)
print(x < 5)
print(x != 3)
```

Expected output:

```
True<br/>False<br/>True
```

Logical operators

`and` is true when both conditions are true; `or` is true when at least one is true; `not` reverses a truth value.

```
age = 19
has_id = True
print(age >= 18 and has_id)
print(age < 18 or has_id)
print(not has_id)
```

Expected output:

```
True<br/>True<br/>False
```

Assignment operators

= assigns a value. Compound forms update an existing variable: `+=`, `-=`, `*=`, and `/=`.

```
score = 10
score += 4
score *= 2
print(score)
```

Expected output:

```
28
```

Step by step: start at 10; add 4 to get 14; multiply 14 by 2 to get 28. Do not confuse = (assignment) with == (comparison).

4. Conditional Statements

Decision-making lets a program choose which instructions to run based on whether a condition is true or false. In Python, the condition is followed by a colon, and the indented lines form the block.

if statement

```
temperature = 30
if temperature > 25:
    print("It is warm.")
```

Expected output:

```
It is warm.
```

Step by step: Python checks whether 30 is greater than 25. The condition is true, so the indented print statement runs.

if-else statement

```
number = 8
if number % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Expected output:

```
Even
```

The remainder after 8 is divided by 2 is 0. Therefore the condition is true and Python runs the first block; the `else` block is skipped.

if-elif-else statement

```
mark = 76
if mark >= 90:
    grade = "A"
elif mark >= 75:
    grade = "B"
elif mark >= 50:
    grade = "C"
else:
    grade = "Needs improvement"
print(grade)
```

Expected output:

```
B
```

Python tests conditions from top to bottom. 76 is not at least 90, but it is at least 75, so `grade` becomes `"B"`. The remaining conditions are not checked.

Syntax reminders

• Use a colon (`:`) after `if`, `elif`, and `else`.

• Indent the statements inside each block consistently (four spaces is conventional).

• Use `==` to compare values, not `=`.

• Conditions can use comparisons and logical operators, such as `age >= 18 and has_id`.

Everyday analogy: if it is raining, take an umbrella; otherwise, leave it at home. The condition decides which path is followed.

5. Loops in Python

A **loop** repeats a block of code. Loops reduce repeated instructions and are useful when processing lists, counting, or repeating work until a condition changes.

for loop and range()

A `for` loop visits items in a sequence. `range(stop)` produces integers from 0 up to, but not including, `stop`. `range(start, stop, step)` lets you choose the starting value and step.

```
for i in range(1, 6):
    print(i)
```

Expected output:

```
1<br/>2<br/>3<br/>4<br/>5
```

The loop assigns 1, 2, 3, 4, and 5 to `i` in turn. The stop value 6 is excluded.

```
for i in range(2, 7, 2):
    print(i)
```

Expected output:

```
2<br/>4<br/>6
```

while loop

A `while` loop repeats while its condition remains true. Make sure the loop can eventually stop; otherwise, it may run forever.

```
count = 1
while count <= 3:
    print(count)
    count += 1
```

Expected output:

```
1<br/>2<br/>3
```

Start with count = 1. Each iteration prints count and adds 1. When count becomes 4, the condition is false and the loop ends.

break and continue

`break` immediately exits the nearest loop. `continue` skips the rest of the current iteration and moves to the next one.

```
for n in range(1, 6):
    if n == 3:
        continue
    if n == 5:
        break
    print(n)
```

Expected output:

```
1<br/>2<br/>4
```

When n is 3, `continue` skips printing it. When n reaches 5, `break` ends the loop. Values 1, 2, and 4 are printed.

Choosing a loop

Use `for` when iterating through a known sequence or range. Use `while` when repetition depends on a condition that may change during execution.

6. Functions in Python

A **function** is a reusable block of code that performs a task. Functions make programs easier to organise, test, and maintain.

Built-in and user-defined functions

Built-in functions are provided by Python, for example `print()`, `len()`, `type()`, and `sum()`. A **user-defined function** is created by the programmer using `def`.

```
numbers = [2, 4, 6]
print(len(numbers))
print(sum(numbers))
print(type(numbers).__name__)
```

Expected output:

```
3<br/>12<br/>list
```

Defining and calling a function

```
def greet():
    print("Hello, student!")

greet()
```

Expected output:

```
Hello, student!
```

The keyword `def` begins a function definition. `greet` is the function name. The indented statement is its body. The call `greet()` runs the body.

Parameters, arguments, and return values

A **parameter** is a name listed in a function definition. An **argument** is the actual value passed when calling the function. `return` sends a result back to the caller.

```
def add(a, b):
    result = a + b
    return result

answer = add(3, 5)
print(answer)
```

Expected output:

```
8
```

Here, `a` and `b` are parameters; 3 and 5 are arguments. The function calculates 8 and returns it. The returned value is stored in `answer`.

Function with a decision

```
def is_adult(age):
    return age >= 18

print(is_adult(20))
print(is_adult(15))
```

Expected output:

```
True<br/>False
```

The expression `age >= 18` evaluates to a Boolean value. That value is returned by the function.

Why use functions?

• Reuse code without copying it repeatedly.

• Give a task a clear name, such as `calculate_total()`.

• Break a large problem into smaller, manageable parts.

• Test one task at a time and make changes more safely.

7. Lists and Tuples

A **list** and a **tuple** are ordered collections that can store multiple values. In both, indexing starts at 0.

Lists

A list is written with square brackets `[]`. Lists are **mutable**, meaning their items can be changed after creation.

```
fruits = ["apple", "banana", "mango"]
 print(fruits[0])
print(fruits[2])
 fruits[1] = "orange"
 fruits.append("grape")
 print(fruits)
 print(len(fruits))
```

Expected output:

```
apple<br/>mango<br/>['apple', 'orange', 'mango', 'grape']<br/>4
```

Index 0 selects the first item; index 2 selects the third. Assignment changes the second item. `append()` adds an item to the end, and `len()` counts the items.

Tuples

A tuple is usually written with parentheses `()`. Tuples are **immutable**, meaning their items cannot be reassigned after creation.

```
point = (4, 7)
 print(point[0])
 print(len(point))
```

Expected output:

```
4<br/>2
```

Lists and tuples: key differences

Feature	List	Tuple
Common notation	[1, 2, 3]	(1, 2, 3)
Can items be reassigned?	Yes (mutable)	No (immutable)
Useful when	The collection needs changes	The collection should stay fixed
Indexing	Starts at 0	Starts at 0

Indexing and basic operations

```
values = [10, 20, 30, 40]
 print(values[1])
 print(values[-1])
 print(20 in values)
 print(values[1:3])
```

Expected output:

```
20<br/>40<br/>True<br/>[20, 30]
```

Positive index 1 selects 20. Index -1 selects the last item. The `in` operator checks membership. Slice `[1:3]` includes index 1 but stops before index 3.

Common error: accessing a list index that does not exist raises `IndexError`. For a collection of length 4, valid positive indexes are 0, 1, 2, and 3.

8. Practice Questions

Try answering without looking at the answer key. Each MCQ has one correct answer.

A. Multiple-choice questions

1. Which keyword defines a function?
A. function B. def C. fun D. make

2. What is the type of 4.5?
A. int B. str C. float D. bool

3. What does 10 % 3 return?
A. 3 B. 1 C. 0 D. 3.33

4. Which operator checks equality?
A. = B. != C. == D. >

5. What is printed by print(2 ** 3)?
A. 5 B. 6 C. 8 D. 9

6. Which collection is mutable?
A. tuple B. list C. int D. str

7. What is the first index of a list?
A. 1 B. -1 C. 0 D. depends on list

8. What does break do inside a loop?
A. Restarts it B. Exits the loop C. Skips one iteration only D. Prints a value

9. What is range(3) equivalent to producing?
A. 1,2,3 B. 0,1,2 C. 0,1,2,3 D. 3 only

10. Which value is Boolean?
A. "True" B. 1.0 C. True D. "False"

B. Short-answer questions

1. What is a variable? Give one example.

2. State two differences between a list and a tuple.

3. Explain the purpose of an if-else statement.

4. What is the difference between a parameter and an argument?

5. What is the difference between a for loop and a while loop?

C. Python programming exercises

1. Write a program to add two numbers and print the result.

2. Write a program that prints whether an integer is even or odd.

3. Use a for loop to print numbers from 1 to 5.

4. Define a function square(n) that returns the square of n; call it with 4.

5. Create a list of three subjects, append one more subject, and print the list.

9. Answer Key, Summary and Revision

A. MCQ answer key with explanations

1. B — def — The def keyword begins a user-defined function.

2. C — float — 4.5 has a decimal part, so its type is float.

3. B — 1 — 10 divided by 3 leaves a remainder of 1.

4. C — == — Double equals compares values; single equals assigns.

5. C — 8 — 2 ** 3 means 2 raised to the power 3: 2 × 2 × 2.

6. B — list — Lists are mutable; tuples are immutable.

7. C — 0 — Python sequences use zero-based indexing.

8. B — Exits the loop — break ends the nearest enclosing loop.

9. B — 0, 1, 2 — range(3) starts at 0 and excludes 3.

10. C — True — True without quotes is a Boolean value; "True" is text.

B. Suggested short answers

1. A variable is a name referring to a value, e.g. age = 18.

2. A list uses [] and can be changed; a tuple commonly uses () and cannot be changed. Both are ordered and indexed from 0.

3. if-else chooses between two code blocks: the if block runs when its condition is true; otherwise the else block runs.

4. A parameter appears in the function definition; an argument is the value supplied when the function is called.

5. A for loop iterates over a sequence/range; a while loop repeats as long as its condition is true.

C. Sample solutions for programming exercises

```
# 1. Add two numbers
 a, b = 4, 6
 print(a + b)                  # Output: 10

# 2. Even or odd
 n = 7
 if n % 2 == 0:
     print("Even")
 else:
    print("Odd")               # Output: Odd

 # 3. Print 1 to 5
 for i in range(1, 6):
    print(i)                   # Output: 1 2 3 4 5 (one per line)

# 4. Square function
 def square(n):
    return n * n
 print(square(4))              # Output: 16

 # 5. List update
subjects = ["Python", "Maths", "Physics"]
 subjects.append("Chemistry")
 print(subjects)
 # Output: ['Python', 'Maths', 'Physics', 'Chemistry']
```

Quick revision notes

• Assignment: = • Equality: == • Remainder: % • Power: **
• Decisions: if / elif / else • Repetition: for / while
• Function: def defines it; return sends a result back.
• List: mutable; tuple: immutable. Indexing begins at 0.

Common beginner mistakes

• Missing colon after a condition, loop, or function definition.
• Incorrect indentation inside a block.
• Using = instead of == in a comparison.
• Forgetting that range() excludes its stop value.
• Creating an infinite while loop by not updating its condition.
• Accessing an index that is outside the collection.

Glossary

Argument: value passed to a function. **Boolean:** True or False. **Condition:** expression evaluated as true/false. **Data type:** category of a value. **Function:** reusable block of code. **Identifier:** name used for a program item. **Index:** position in a sequence. **Iteration:** one pass through a loop. **Keyword:** reserved word. **Loop:** repeated execution. **Parameter:** name in a function definition. **Return value:** result sent back by a function. **Syntax:** rules for writing valid code.